

TECHWIZ 7 — THE WORLD TECH CHAMPIONSHIP

Category: Data Science Intelligence Arena

DineIQ Analytics

Team Name: IntelliData

Team Members:

Muhammad Kaif Student1545319,

Saba Asif Student1545228 ,

Mahad Ali Khan Student1545320,

Fatima Sehar Student1526925,

Category: Data Science

Institute: Aptech Latifabad

Mentor / Faculty: Miss Imama

Center Manager: Sir Shohaib

Academic Head: Miss Imama

Submission Date: 29 September 2026

Acknowledgement

We would like to express our sincere gratitude to Aptech Limited and the TechWiz 7 — The World Tech Championship organizing team for providing us the opportunity and platform to design and develop the DineIQ Analytics application under the Data Science Intelligence Arena category.

We are thankful to our mentor/faculty, Miss Imama, for continuous guidance, technical direction, and encouragement throughout the five days of development. We also acknowledge the support of our institute, Aptech Latifabad, for providing the infrastructure required to build and test this Big Data and Data Science solution.

Finally, we thank every member of our team for their dedication, collaborative effort, and individual contribution to the Big Data pipeline, the Python Data Science pipeline, and the Web dashboard that together make up DineIQ Analytics.

Table of Contents

Acknowledgement.....	2
Table of Contents.....	3
1. Problem Definition.....	5
2. Background and Business Necessity.....	5
3. Proposed Solution.....	5
4. Purpose of the Document.....	5
5. Scope of the Project.....	6
6. Assumptions.....	6
7. Constraints.....	6
8. Functional Requirements.....	6
9. Non-Functional Requirements.....	7
10. System Architecture.....	7
11. Big Data Architecture.....	8
12. Module Descriptions.....	9
13. Database Design.....	9
14. Entity Relationship Design.....	9
15. Data Dictionary (Sample).....	10
16. Process Diagrams.....	10
16.1 Data Flow Diagram (Level 0).....	10
16.2 Use Case Diagram (Key Actors and Use Cases).....	10
16.3 Activity Diagram (Menu Classification Flow).....	11
16.4 Sequence Diagram (Dual-Pipeline Prediction Request).....	12
17. Analytical Workflow.....	13
18. Dataset-Generation Methodology.....	13
19. Data-Quality Methodology.....	13
20. Spark Processing Pipeline.....	14
21. Feature Engineering.....	14
22. Menu Classification Methodology.....	14
23. Customer Segmentation.....	14
24. Market-Basket Analysis.....	14
25. Demand Forecasting.....	15
26. Wastage Analysis.....	15
27. Pricing Analysis.....	15

28. Promotion Analysis.....	15
29. Anomaly Detection.....	15
30. Spark MLlib Model Design.....	15
31. Python Data Science Model Design.....	15
32. Dual-Pipeline Result Comparison.....	16
33. Model Evaluation.....	16
34. Testing Strategy.....	16
35. Security Considerations.....	16
36. Privacy Considerations.....	16
37. Limitations.....	16
38. Future Enhancements.....	17
39. Hardware and Software Requirements.....	17
39.1 Hardware.....	17
39.2 Software.....	17
40. Team Contribution Record.....	17
41. References.....	18

1. Problem Definition

Restaurant businesses generate large volumes of data through customer orders, menu items, pricing, ratings, promotions, inventory consumption, food wastage, ordering channels, and restaurant locations. Traditional analysis methods — basic reports, spreadsheets, and manual comparison of sales figures — fail to reveal the complex relationships between menu performance, profitability, customer behaviour, promotions, ratings, wastage, and time-based ordering patterns.

DineIQ Analytics addresses this gap by applying Big Data processing and Data Science techniques to automatically identify profitable items, slow-moving dishes, peak ordering periods, customer segments, wastage patterns, and pricing/promotion effects, and by generating evidence-backed, actionable recommendations for restaurant managers.

2. Background and Business Necessity

Restaurant chains today operate across multiple locations and channels (dine-in, takeaway, website/app, third-party delivery), each generating its own stream of transactional and operational data. Without an integrated analytics layer, managers rely on intuition or isolated sales reports, which can misclassify a dish's success (e.g. a high-selling dish that is actually loss-making) and lead to poor decisions on pricing, promotions, and inventory.

DineIQ Analytics is necessary because it converts raw, multi-source restaurant data into a single, trustworthy intelligence layer — validated by two independent analytical pipelines — that supports faster and more reliable operational decisions.

3. Proposed Solution

DineIQ Analytics is a Big Data and Data Science-powered, Web-based restaurant intelligence platform. It ingests data related to orders, menu items, customers, restaurant locations, pricing, ratings, promotions, inventory, and food wastage, and processes it using Apache Spark, PySpark, and Spark SQL.

Every menu item is scored using multiple indicators (sales volume, revenue, cost, contribution margin, ratings, repeat purchases, wastage, promotion dependency, and demand trend) and classified into one of four categories: Profit Driver, Volume Driver, Hidden Opportunity, or Low Performer.

To make results independently verifiable, two separate Data Science pipelines are developed on the same underlying dataset:

- Big Data Pipeline — Apache Spark, PySpark, Spark SQL, and Spark MLlib
- Python Data Science Pipeline — Python, Pandas, Scikit-learn, XGBoost (or equivalent)

The application compares the outputs of both pipelines, reports matches, mismatches, and numerical differences, and presents all business insights, predictions, alerts, classifications, forecasts, and recommendations through an interactive Web dashboard.

4. Purpose of the Document

This document defines the design, functionality, and implementation plan of DineIQ Analytics. It provides a shared understanding of the project objectives for the team, evaluators, mentors, and any stakeholder reviewing the submission, and acts as a structured roadmap for development, testing, and deployment.

5. Scope of the Project

The scope covers the design and development of a Web-based restaurant intelligence application that processes order, menu, pricing, customer, rating, promotion, inventory, and wastage data using Apache Spark, PySpark, Spark SQL, and Machine Learning. It includes menu profitability intelligence, customer segmentation, market-basket analysis, demand forecasting, wastage intelligence, pricing intelligence, promotion analytics, multi-location comparison, anomaly detection, a recommendation engine, and what-if analysis.

Integration with live POS systems, payment gateways, third-party delivery platforms, or enterprise restaurant management systems is outside the scope of this project.

6. Assumptions

- A realistic, synthetic restaurant dataset is generated by the team since no ready-made competition dataset is provided.
- Evaluators will introduce a hidden dataset and a surprise modification during final evaluation.
- Users (managers, analysts, administrators) have basic familiarity with restaurant KPIs and dashboards.
- The application runs in a single-node or small-cluster Spark environment sufficient for the specified dataset scale.

7. Constraints

The project's effectiveness depends on the quality, completeness, consistency, and accuracy of the generated dataset, and on the computational resources available for Big Data processing. Differences may occur between the Spark/PySpark pipeline and the independent Python pipeline for the same analytical task. Privacy and security of customer, transaction, and business data must be maintained in line with applicable data-protection practices.

8. Functional Requirements

The key functional modules of DineIQ Analytics are summarised below (see the SRS for the complete list of 66 functional requirements).

#	Module	Description
1	User & Access	Secure login and role-based access control for managers, analysts, regional managers, and administrators.
2	Master Data Management	Restaurant locations, menu items/categories, pricing history, customers, orders, promotions, ratings, inventory, and wastage.
3	Big Data Ingestion	Spark/PySpark ingestion with schema validation, data-quality checks, and cleaning.

#	Module	Description
4	Feature Engineering	Revenue, cost, margin, RFM, wastage %, promotion dependency, and related derived features.
5	Menu Intelligence	Profitability analysis and classification into Profit Driver / Volume Driver / Hidden Opportunity / Low Performer.
6	Customer Intelligence	RFM analysis, segmentation, and churn-risk identification.
7	Market-Basket Analysis	Association-rule mining (support, confidence, lift) and bundle/cross-sell recommendations.
8	Forecasting	Time-aware demand forecasting with MAE / RMSE / MAPE evaluation.
9	Wastage Intelligence	Wastage analysis and wastage-risk prediction.
10	Pricing & Promotion Intelligence	Price-sensitivity analysis, promotion-effectiveness analysis, and promotion-trap detection.
11	Anomaly Detection	Sales anomalies and rating anomalies.
12	Dual-Pipeline ML	Spark MLlib and Python models trained independently and compared.
13	Recommendation Engine	Evidence-based, prioritised recommendations with supporting data.
14	What-If Analysis	Scenario simulation for price, discount, promotion, and demand changes.
15	Dashboards & Reports	Executive, Menu, Customer, Wastage, Forecast, and Dual-Pipeline dashboards; downloadable reports and data export.

9. Non-Functional Requirements

Attribute	Requirement
Performance	Predictions from both pipelines returned within 5 seconds under normal load.
Scalability	Architecture supports scaling to at least 5,000,000 order-line records without redesign.
Usability	Intuitive, responsive Web interface for managers, analysts, regional managers, and administrators.
Accuracy	Classification models: $\geq 85\%$ test accuracy or macro F1 ≥ 0.80 . Forecasts must beat a simple baseline.
Availability	$\geq 99\%$ uptime during evaluation periods.
Security & Privacy	Role-based access, anonymised customer data, and an audit trail of processing jobs and admin actions.

10. System Architecture

DineIQ Analytics follows a layered Big Data architecture: (1) Restaurant Data Sources → (2) Data Ingestion Layer → (3) Big Data Storage (CSV/JSON/Parquet) → (4) Apache Spark/PySpark processing → (5) Data Cleaning & Validation → (6) Spark SQL Processing → (7) Feature Engineering → (8a) Spark/MLlib Analytics and (8b) Independent Python Analytics, run in parallel → (9) Result Comparison → (10) Restaurant Intelligence layer (profitability, wastage, basket analysis, peak

periods, pricing, customer behaviour) → (11) Recommendation Engine → (12) Web Dashboard and Reports.

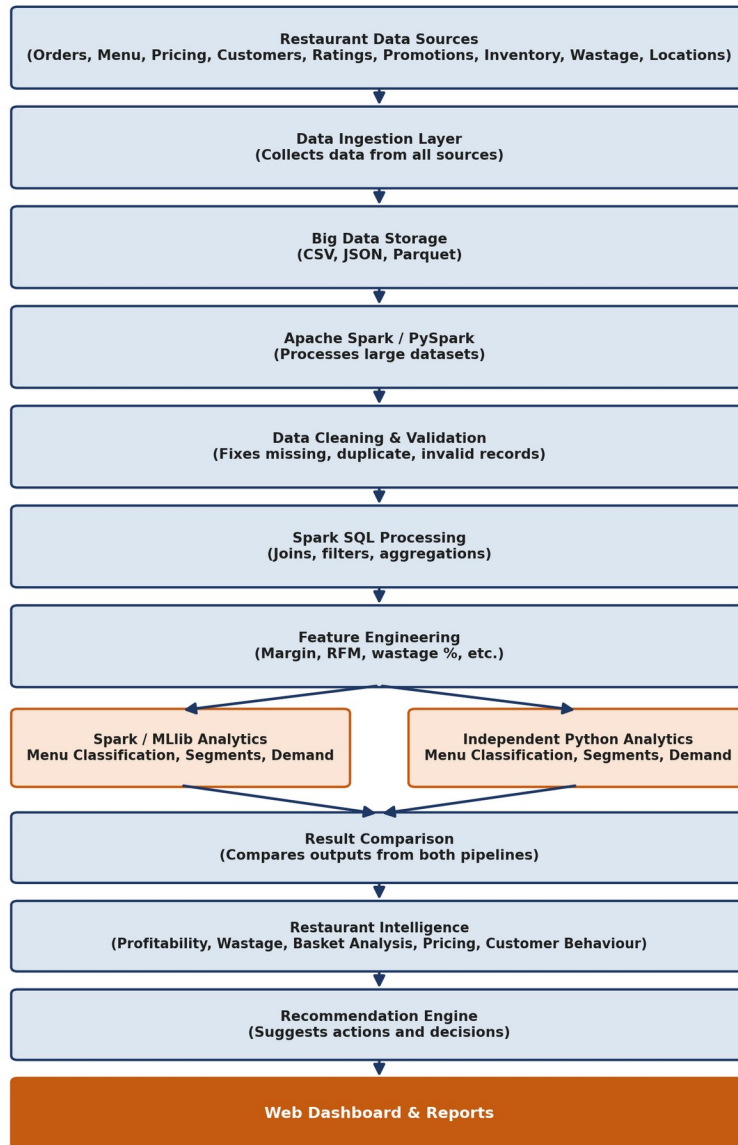


Figure 1: System / Big Data Architecture of DineIQ Analytics

11. Big Data Architecture

Raw restaurant data (Customers, Orders, Order_Items, Menu_Items, Menu_Categories, Restaurants, Pricing_History, Promotions, Ratings, Inventory, Wastage) is ingested with Spark/PySpark using explicit schema definitions and partition handling. At least one large, processed dataset is persisted in Parquet format for efficient columnar storage and downstream querying via Spark SQL. Data quality checks (missing values, duplicates, invalid prices/dates/ratings, cancelled transactions) run before cleaning and integration, and every cleaning decision is logged.

12. Module Descriptions

Module 1 — Data Management: Stores and maintains restaurant locations, menu, pricing history, customers, orders, promotions, ratings, inventory, and wastage records with appropriate primary/foreign keys.

Module 2 — Big Data Processing: Handles ingestion, schema validation, data-quality assessment, cleaning, Spark SQL joins/aggregations, and Parquet storage.

Module 3 — Feature Engineering & Analytics: Derives business features (margin, RFM, wastage %, etc.) and performs menu classification, segmentation, basket analysis, forecasting, wastage/pricing/promotion analysis, and anomaly detection.

Module 4 — Machine Learning: Trains and evaluates at least three Spark MLlib models and an independent set of Python (Scikit-learn/XGBoost) models for the same analytical task(s), then compares their outputs.

Module 5 — Recommendation Engine: Converts analytical findings into prioritised, evidence-backed recommendations (menu, pricing, promotion, inventory, customer targeting).

Module 6 — Web Dashboard: Presents the Executive, Menu, Customer, Wastage, Forecast, and Dual-Pipeline dashboards, with search/filtering, downloadable reports, and data export.

13. Database Design

The application uses a relational schema for master and transactional data (e.g. PostgreSQL/MySQL/SQLite) alongside file-based Big Data storage (CSV/JSON/Parquet) for large analytical datasets. Core tables and their primary keys:

Table	Primary Key	Key Foreign Keys
Customers	customer_id	—
Orders	order_id	customer_id, location_id, channel_id, promotion_id
Order_Items	order_item_id	order_id, item_id
Menu_Items	item_id	category_id
Menu_Categories	category_id	—
Restaurants (Locations)	location_id	—
Pricing_History	price_id	item_id
Promotions	promotion_id	applicable item_id(s)
Ratings	rating_id	item_id, location_id, customer_id
Inventory	inventory_id	item_id, location_id
Wastage	wastage_id	item_id, location_id

14. Entity Relationship Design

Customers place Orders (1-to-many). Each Order has one or more Order_Items (1-to-many), each referencing one Menu_Item. Menu_Items belong to a Menu_Category (many-to-1) and have a time-

series of Pricing_History records (1-to-many). Orders occur at a Restaurant location and may apply a Promotion (many-to-1 each). Ratings, Inventory, and Wastage each reference a Menu_Item and a Restaurant location.

Draw the full ER diagram (all 11 tables with PK/FK arrows) in a diagramming tool such as draw.io / dbdiagram.io and insert it here before final submission.

15. Data Dictionary (Sample)

Field	Table	Type	Description
order_id	Orders	String/Int	Unique order identifier
customer_id	Customers	String/Int	Unique customer identifier
item_id	Menu_Items	String/Int	Unique menu item identifier
location_id	Restaurants	String/Int	Unique restaurant/location identifier
Quantity	Order_Items	Integer	Units of the item ordered
unit_price	Order_Items	Decimal	Selling price at time of order
cost_price	Menu_Items	Decimal	Preparation/ingredient cost
contribution_margin	Derived	Decimal	(Revenue – Cost) / Revenue
wastage_qty	Wastage	Decimal	Quantity wasted
rating_value	Ratings	Decimal (1-5)	Customer rating score

Extend this table to cover every field across all 11 dataset tables in the final submission.

16. Process Diagrams

16.1 Data Flow Diagram (Level 0)

Restaurant Data Sources → Data Ingestion (Spark) → Data Quality & Cleaning → Feature Store → [Spark MLlib Pipeline | Python Pipeline] → Result Comparison → Recommendation Engine → Web Dashboard / Reports.

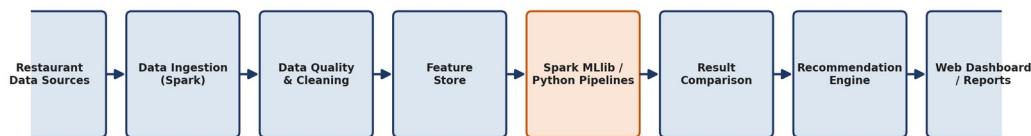


Figure 2: Level-0 Data Flow Diagram

16.2 Use Case Diagram (Key Actors and Use Cases)

- Administrator: Manage locations, menu, users, roles; monitor Spark jobs; view audit trail.

- Restaurant Manager: View dashboards, recommendations, what-if analysis, download reports.
- Analyst: Run/inspect Spark & Python pipelines, review dual-pipeline comparison, export data.
- Regional Manager: Compare locations, review multi-location intelligence.

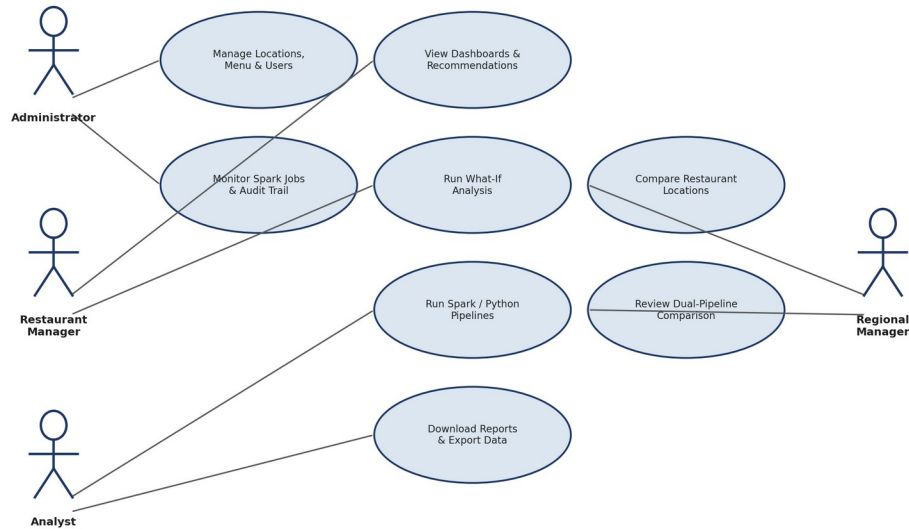


Figure 3: Use Case Diagram

16.3 Activity Diagram (Menu Classification Flow)

Load menu & transaction data → Compute indicators (sales, revenue, margin, rating, repeat rate, wastage, promotion dependency) → Apply classification model → Assign category (Profit Driver / Volume Driver / Hidden Opportunity / Low Performer) → Store result → Trigger recommendation if Low Performer or Hidden Opportunity.

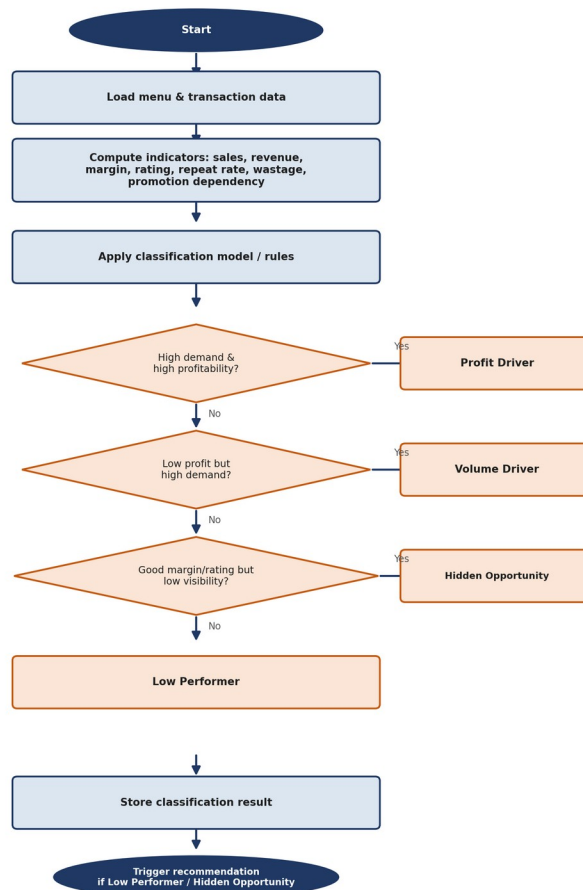


Figure 4: Activity Diagram — Menu Performance Classification

16.4 Sequence Diagram (Dual-Pipeline Prediction Request)

User → Dashboard: request prediction. Dashboard → Spark Service: run Spark MLlib model. Dashboard → Python Service: run Python model (in parallel). Both services → Comparison Engine: return results. Comparison Engine → Dashboard: match/mismatch, agreement %. Dashboard → User: display comparison.

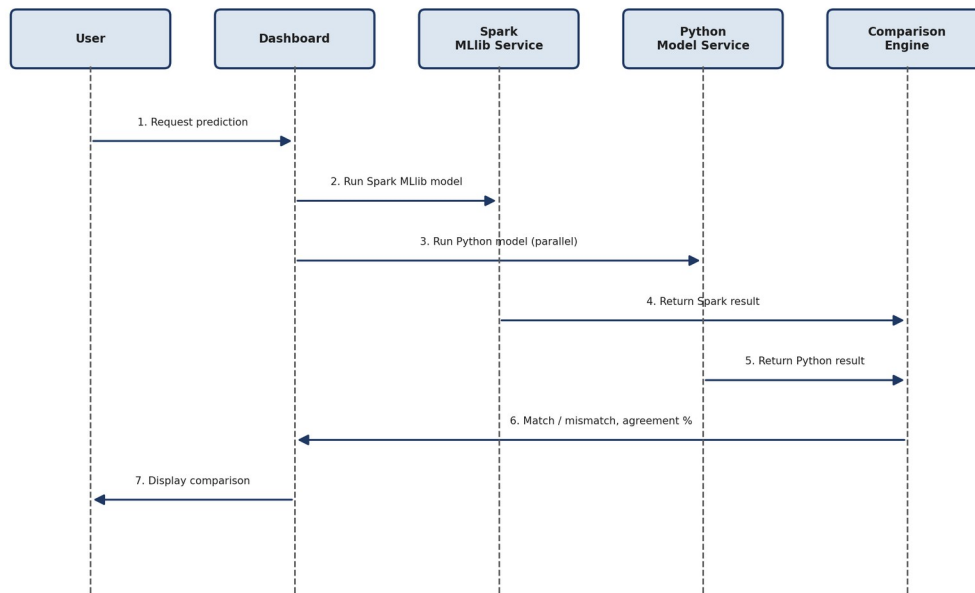


Figure 5: Sequence Diagram — Dual-Pipeline Prediction Request

17. Analytical Workflow

The end-to-end analytical workflow follows the 50 development steps defined in the SRS: dataset creation → Big Data storage → Spark ingestion → data-quality assessment → cleaning → integration → feature engineering → EDA → menu profitability → menu classification → tricky-case handling → Spark MLlib modelling → independent Python modelling → dual-pipeline verification → customer segmentation & RFM → market-basket analysis → bundle recommendations → peak-period analysis → demand forecasting → forecast evaluation → wastage analysis & prediction → price intelligence & sensitivity → promotion effectiveness & trap detection → rating analysis & anomaly detection → sales anomaly detection → slow-moving dish detection → multi-location & channel analysis → churn-risk identification → recommendation engine → what-if analysis → dashboards.

18. Dataset-Generation Methodology

A synthetic but realistic restaurant dataset is generated programmatically to meet the minimum scale required by the SRS: at least 1,000,000 order-line records, 100,000 orders, 50,000 customers, 150 menu items across 10+ categories, 20+ locations, 12+ months of history, 100,000 ratings, 50,000 wastage records, multiple pricing revisions, and multiple promotion campaigns.

The generator deliberately injects realistic complexity — missing values, duplicate records, cancelled orders, seasonal and weekend demand patterns, multi-location differences, high-value and churned customers, popular-but-low-margin dishes, profitable-but-low-selling dishes, rating/sales anomalies, and misleading promotions — so that downstream data-quality, cleaning, and modelling steps have genuine issues to detect and resolve.

19. Data-Quality Methodology

Before analysis, the pipeline profiles the data for missing values, duplicate orders/order-lines, invalid prices, negative quantities, invalid dates/ratings, missing or invalid IDs, impossible wastage quantities, incorrect discounts, and inconsistent units, producing a Data Quality Report. Records are

then corrected, removed, or quarantined according to documented rules, and every cleaning decision is logged for auditability.

20. Spark Processing Pipeline

Apache Spark/PySpark is used for ingestion (explicit schema + schema inference), large/multi-file loading, and partitioning. Spark SQL performs the complex joins (Orders–Customers, Orders–Order_Items, Order_Items–Menu_Items, Menu_Items–Categories/Pricing/Ratings/Inventory/Wastage, Orders–Locations/Promotions) and aggregations needed for feature engineering and EDA. At least one processed dataset is persisted in Parquet format.

21. Feature Engineering

Derived features include item revenue, cost, contribution margin, profit %, order frequency, popularity, repeat-purchase rate, average rating and rating trend, wastage %, promotion dependency, discount %, customer recency/frequency/monetary value, average order value, peak-hour frequency, weekend-order ratio, location performance, channel preference, basket size, and price-change %.

22. Menu Classification Methodology

Each menu item is scored on quantity sold, revenue, cost, contribution margin, rating, repeat-purchase rate, wastage %, promotion dependency, and sales trend, then classified as Profit Driver, Volume Driver, Hidden Opportunity, or Low Performer using a data-driven rule/model rather than a single hard-coded field. The methodology explicitly handles tricky cases such as a high-selling but loss-making dish, a highly profitable but rarely purchased dish, a promotion-dependent dish, and a dish that performs differently across locations or only on weekends.

23. Customer Segmentation

Customers are segmented using RFM (Recency, Frequency, Monetary value) together with average order value, favourite categories, promotion sensitivity, channel, and time-of-day preference, into segments such as High-Value Loyal, Frequent, Promotion-Driven, At-Risk, New, and Occasional customers.

24. Market-Basket Analysis

Association-rule mining identifies frequently co-purchased items and computes support, confidence, and lift for each item pair/combination. Rules meeting minimum thresholds feed the bundle, cross-sell, and upsell recommendations shown on the dashboard, with the underlying association-rule evidence attached to each suggestion.

25. Demand Forecasting

Demand is forecast per menu item, category, and location for configurable future periods. Time-aware validation is enforced: training data represents earlier periods and test data represents strictly later, unseen periods, preventing any leakage of future information into the model. Forecast accuracy is evaluated using MAE, RMSE, MAPE, and R^2 where appropriate, benchmarked against a simple baseline forecast.

26. Wastage Analysis

Wastage is analysed by menu item, category, location, day, time period, demand, inventory consumption, and promotion status. A wastage-risk prediction model estimates the likelihood of high wastage for upcoming periods using historical demand/wastage, day of week, season, location, promotion activity, and forecast demand as predictive variables.

27. Pricing Analysis

Price intelligence examines the relationship between price, demand, revenue, contribution margin, discount, rating, and repeat purchase. Items are classified as Highly Price Sensitive, Moderately Price Sensitive, or Low Price Sensitivity based on historical price-change and order-behaviour evidence, supporting repricing recommendations.

28. Promotion Analysis

Promotions are evaluated on order volume, revenue, contribution margin, customer acquisition, repeat purchase, average order value, wastage, and post-promotion behaviour — a sales increase alone is not treated as success. Promotion-trap detection flags cases such as sales rising while profit falls, margin collapsing as customer count grows, promotions increasing wastage, or purchases that stop once the discount ends.

29. Anomaly Detection

Sales anomaly detection flags sudden spikes/drops, abnormally high order values, unusual discounts, and duplicate transactions. Rating anomaly detection flags sudden rating spikes/drops, excessive identical ratings, unusually high rating volume in a short period, and ratings inconsistent with purchasing behaviour.

30. Spark MLlib Model Design

At least three Spark MLlib algorithms (e.g. Logistic Regression, Random Forest, Gradient-Boosted Trees, or K-Means for clustering tasks) are trained and compared for the selected analytical task(s) — such as menu classification, customer segmentation, or demand/wastage-risk prediction — with the best model selected using appropriate evaluation metrics.

31. Python Data Science Model Design

An independent Python pipeline (Pandas, NumPy, Scikit-learn, XGBoost, or similar) processes the same underlying restaurant records for the chosen task(s), without importing or reusing Spark-generated predictions, ensuring the two pipelines produce genuinely independent results.

32. Dual-Pipeline Result Comparison

For at least one major analytical task, Spark and Python results are compared record-by-record on 100+ unseen cases. The comparison report includes Record ID, actual class/value, Spark result, Python result, match/mismatch, numerical difference (where applicable), an explanation of disagreements, and an overall agreement percentage. Exact equality is not required, since the models are trained independently.

33. Model Evaluation

Classification models are evaluated with accuracy, precision, recall, F1-score, and confusion matrices where applicable, targeting $\geq 85\%$ accuracy or macro F1 ≥ 0.80 . Regression/forecasting models are evaluated with MAE, RMSE, MAPE, and R^2 , and must beat a simple baseline forecast.

34. Testing Strategy

Testing covers functional and integration tests, Big Data ingestion and schema-validation tests, data-quality tests, Spark transformation and Spark SQL tests, Spark and Python model tests, dual-pipeline comparison tests, forecast/basket/wastage/promotion/pricing/anomaly tests, security and boundary tests, and hidden-data readiness tests. Difficult business cases (e.g. high-selling loss-making dish, promotion that increases sales but reduces profit, Spark/Python disagreement case) are specifically exercised.

35. Security Considerations

Access is controlled through role-based authentication for managers, analysts, regional managers, and administrators. All data-processing jobs, predictions, exports, and administrative actions are logged in an audit trail, and the system surfaces clear, understandable errors for processing, model, Spark, and database failures.

36. Privacy Considerations

Customer data is anonymised in storage and analysis wherever identifying details are not required for the analytical task, and access to raw customer-linked records is restricted by role. Data handling follows applicable data-protection practices given the constraints of the competition environment.

37. Limitations

- Results depend on the quality, realism, and balance of the synthetically generated dataset.
- Spark and Python pipelines, being trained independently, will not always agree exactly.
- Computational resources available during the competition may limit the practical dataset size processed locally.
- The application does not integrate with live POS, payment, or third-party delivery systems.

38. Future Enhancements

- Real-time streaming ingestion (e.g. Kafka) for live order and wastage data.
- Integration with POS and third-party delivery platforms for automated data capture.
- Deep-learning-based demand forecasting (e.g. LSTM/Temporal models) for improved accuracy.
- Automated A/B testing framework for promotion and pricing experiments.
- Mobile app for on-the-go manager alerts and recommendations.

39. Hardware and Software Requirements

39.1 Hardware

- Intel Core i5/i7 processor or higher

- 8/16 GB RAM or higher (recommended for local Spark workloads)
- Colour SVGA monitor
- 500 GB or more free disk space
- Stable internet connection

39.2 Software

- Frontend: HTML5, CSS3, JavaScript, Bootstrap / Streamlit / React
- Backend: Flask / Django / FastAPI / Streamlit
- Big Data Processing: Apache Spark, PySpark, Spark SQL
- Data Store: CSV, JSON, Parquet, Hadoop-compatible storage
- Database: PostgreSQL / MySQL / MongoDB / SQLite / Firebase
- Data Science: Spark MLlib, Scikit-learn, XGBoost, Statsmodels, NumPy, Pandas, SciPy
- Visualization: Plotly, Matplotlib, Streamlit charts / Power BI / Tableau
- Version Control: Git and GitHub
- Deployment: Render / Railway / PythonAnywhere / Streamlit Community Cloud

40. Team Contribution Record

Team Member	Modules Assigned
Muhammad kaif	[e.g. Dataset generation, Spark ingestion]
Saba Asif	[e.g. Data quality, cleaning, Spark SQL]
Mahad Ali Khan	[e.g. Feature engineering, Spark MLlib]
Fatima sehar	[e.g. Python pipeline, dual-pipeline comparison]
Muhammad Kaif	[e.g. Web dashboard, recommendation engine]

41. References

- Reference application (concept only, not to be copied): <https://incentivio.com/feature/menu-intelligence/>
- DineIQ Analytics — Software Requirements Specification, Version 1.0, TechWiz 7 (Aptech Limited).